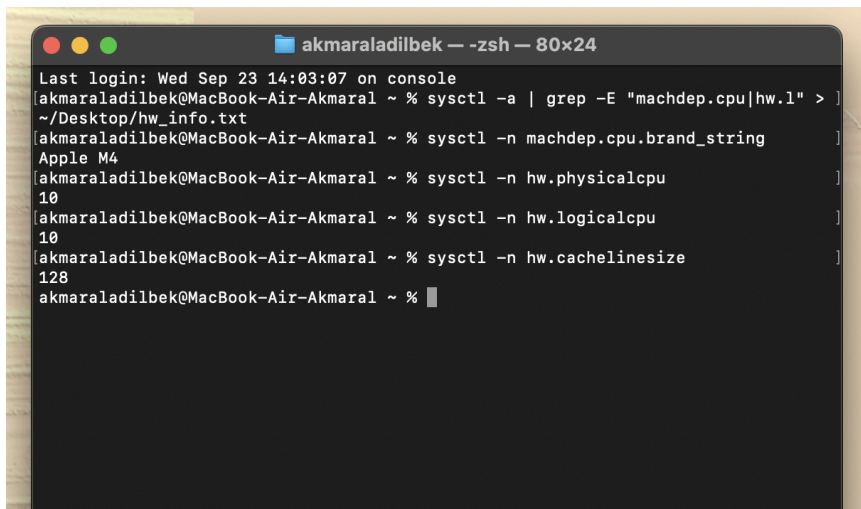


Student Full Name: Akmaral Adilbek
Student ID Number: 230103136
Institutional Email: 230103136@sdu.edu.kz
Practicum Date: 24 September 2026
Laptop Model & OS: MacBook Air 13-inch (M4, 2025), macOS
CPU Model: Apple M4
Physical Cores: 10
Logical Threads: 10
Target Workload Size N: 13,136,000 by last 4 numbers of ID
Cache Line Size: 128 bytes

Phase 1

$N = 10,000,000 + (3136 \times 1,000) = 13,136,000$

Hardware evidence



```
akmaraladilbek — zsh — 80x24
Last login: Wed Sep 23 14:03:07 on console
akmaraladilbek@MacBook-Air-Akmaral ~ % sysctl -a | grep -E "machdep.cpu|hw.1" > ~/Desktop/hw_info.txt
akmaraladilbek@MacBook-Air-Akmaral ~ % sysctl -n machdep.cpu.brand_string
Apple M4
akmaraladilbek@MacBook-Air-Akmaral ~ % sysctl -n hw.physicalcpu
10
akmaraladilbek@MacBook-Air-Akmaral ~ % sysctl -n hw.logicalcpu
10
akmaraladilbek@MacBook-Air-Akmaral ~ % sysctl -n hw.cachelinesize
128
akmaraladilbek@MacBook-Air-Akmaral ~ %
```

Phase 2

Sequential benchmark

Run	Execution Time
Run 1 (Cold/Warm-up)	1.481010 s
Run 2	1.475303 s
Run 3	1.479221 s
Average T _{seq} (Run 2 & 3)	1.477262 s

```

akmaraladilbek@MacBook-Air-Akmaral ~ % ./collatz_parallel 1
./collatz_parallel 1
./collatz_parallel 1
Threads: 1
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 1.469273 seconds
Threads: 1
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 1.444669 seconds
Threads: 1
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 1.444727 seconds
akmaraladilbek@MacBook-Air-Akmaral ~ %

```

Phase 3

```

akmaraladilbek@MacBook-Air-Akmaral ~ % ./collatz_parallel 2
./collatz_parallel 2
./collatz_parallel 2
Threads: 2
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.790554 seconds
Threads: 2
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.764919 seconds
Threads: 2
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.761575 seconds
akmaraladilbek@MacBook-Air-Akmaral ~ %

```

Threads (k)	Run 1 Cold (s)	Run 2 (s)	Run 3 (s)	Avg T _□ (s)	S _{emp} (k) ()	S _{theo} (k) ()	Delta
1	1.469273	1.444669	1.444727	1.444698	1.023	1.000	-0.023
2	0.790554	0.764919	0.761575	0.763247	1.935	1.935	0.000
4	0.448411	0.429666	0.429131	0.429399	3.440	3.636	0.196
8	0.270595	0.263672	0.266179	0.264926	5.576	6.487	0.911
10	0.225388	0.230703	0.228688	0.229696	6.431	7.693	1.261

```

Threads: 4
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.448411 seconds
Threads: 4
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.429666 seconds
Threads: 4
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.429131 seconds
Threads: 8
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.270595 seconds
Threads: 8
N: 13136000

```

```

Threads: 8
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.266179 seconds
Threads: 10
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.225388 seconds
Threads: 10
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.230703 seconds
Threads: 10
N: 13136000
Maximum stopping time: 688
Checksum: 76679673
Execution time: 0.228688 seconds

```

Phase 4

Experiment A

Raw benchmark results

Run	Naive False Sharing	OpenMP Reduction	Penalty Ratio
Run 1	0.324395 s	0.229965 s	1.411×
Run 2	0.314554 s	0.228592 s	1.376×
Run 3	0.306585 s	0.227241 s	1.349×
Average	0.315178 s	0.228599 s	=1.38×

False sharing results

Implementation Variant	Thread Count	Execution Time	Effective Throughput	Speedup Penalty Ratio
Naive hits[tid]++	10	0.315178 s	=41.68 million iter/s	=1.38×
OpenMP Reduction	10	0.228599 s	=57.46 million iter/s	1.00×

```

akmaraladilbek@MacBook-Air-Akmaral ~ % nano false_sharing.c
akmaraladilbek@MacBook-Air-Akmaral ~ % clang -O2 -Xpreprocessor -fopenmp \
-I/opt/homebrew/opt/libomp/include \
-L/opt/homebrew/opt/libomp/lib \
-lomp false_sharing.c -o false_sharing
akmaraladilbek@MacBook-Air-Akmaral ~ % ./false_sharing
./false_sharing
./false_sharing
Threads: 10

Variant 1 - Naive False Sharing
Hits: 10585482
Execution time: 0.324395 seconds
Throughput: 40493819.95 iter/sec

Variant 2 - OpenMP Reduction
Hits: 10585482
Execution time: 0.229965 seconds
Throughput: 57121742.99 iter/sec

Speedup Penalty Ratio: 1.411x
Threads: 10

Variant 1 - Naive False Sharing

```

```

Variant 1 - Naive False Sharing
Hits: 10585482
Execution time: 0.314554 seconds
Throughput: 41760686.70 iter/sec

Variant 2 - OpenMP Reduction
Hits: 10585482
Execution time: 0.228592 seconds
Throughput: 57464848.54 iter/sec

Speedup Penalty Ratio: 1.376x
Threads: 10

Variant 1 - Naive False Sharing
Hits: 10585482
Execution time: 0.306585 seconds
Throughput: 42846182.47 iter/sec

Variant 2 - OpenMP Reduction
Hits: 10585482
Execution time: 0.227241 seconds
Throughput: 57806459.79 iter/sec

```

Experiment B

OpenMP Scheduling Clause	Chunk Size	Average Execution Time	Observed Behavior
schedule(static)	Default (N/k)	0.234690 s	Static partitioning was simple, but the irregular workload can create imbalance between threads.
schedule(static,1000)	1,000	0.223090 s	Round-robin chunks improved workload distribution compared with default static.
schedule(dynamic,100)	100	0.216832 s	It produced the lowest measured average time and distributed irregular work effectively.
schedule(dynamic,10000)	10,000	0.217476 s	Very similar performance to dynamic(100), with larger chunks.

schedule(guided)	Exponential decay	0.219653 s	Adaptive scheduling performed well but was slightly slower than the dynamic configurations in these measurements.
------------------	-------------------	------------	---

```

Threads: 10
static default      Time: 0.255687 s   Checksum: 76679673
static 1000         Time: 0.224262 s   Checksum: 76679673
dynamic 100         Time: 0.217064 s   Checksum: 76679673
dynamic 10000       Time: 0.218384 s   Checksum: 76679673
guided             Time: 0.223409 s   Checksum: 76679673
Threads: 10
static default      Time: 0.224029 s   Checksum: 76679673
static 1000         Time: 0.220821 s   Checksum: 76679673
dynamic 100         Time: 0.217072 s   Checksum: 76679673
dynamic 10000       Time: 0.217549 s   Checksum: 76679673
guided             Time: 0.218880 s   Checksum: 76679673
Threads: 10
static default      Time: 0.224355 s   Checksum: 76679673
static 1000         Time: 0.224186 s   Checksum: 76679673
dynamic 100         Time: 0.216361 s   Checksum: 76679673
dynamic 10000       Time: 0.216495 s   Checksum: 76679673
guided             Time: 0.216670 s   Checksum: 76679673
akmaraladilbek@MacBook-Air-Akmaral ~ %

```

Phase 5

Q1. Micro-Architectural Root Cause of False Sharing

In Variant 1, different threads updated counters that were located close to each other in memory. My CPU has a 128-byte cache line, so several counters can be stored in the same cache line.

When different cores update the same cache line, it creates extra cache invalidation traffic because the cores need to keep their cache data consistent. Because of this, the naive version was slower. Its average time was 0.315178 s, while the reduction version took 0.228599 s.

Q2. Hyperthreading (SMT) Saturation & Physical Core Ceilings

My Apple M4 has 10 physical cores and 10 logical threads, so there are no extra SMT threads on my system.

With 10 threads, I got an empirical speedup of $6.431\times$, not the ideal $10\times$. This happens because threads still share resources such as cache and memory bandwidth, and parallel execution also has synchronization overhead.

Q3. Empirical vs. Theoretical Amdahl Discrepancy

My calculated parallel fraction was $p = 0.9667$, or about 96.67%.

As I increased the number of threads, the difference between theoretical and real speedup became larger. For example, with 10 threads the theoretical speedup was $7.693\times$, but my real result was $6.431\times$.

This difference happens because Amdahl's basic model does not include real hardware problems such as cache/memory contention and synchronization overhead.

Q4. The Scheduling Trade-Off Dilemma

In my experiment, dynamic(100) had the lowest average time at 0.216832 s. dynamic(10000) was very close at 0.217476 s, while default static scheduling was slower at 0.234690 s.

For my machine, dynamic(100) gave a good balance between distributing the work and scheduling overhead. There was no clear point where queue contention became worse than the benefit of workload balancing.

Conclusion

In this lab, I compared sequential and parallel execution of the Collatz workload. My sequential time was 1.477262 s, while using 10 threads reduced the average time to 0.229696 s.

The experiments also showed that real performance is not the same as theoretical performance. False sharing made the program slower, and different scheduling methods also changed the execution time. Overall, the results show that cache behavior, workload balance, and scheduling are important for parallel performance.

